

Deepfake Detection: An Artifact Based Approach

1st Abhimanyu Karki

Dept. of Computer Science
Texas Tech University
Lubbock, Texas, United States
abhi.karki@ttu.edu

2nd Shivansh Sawant

Dept. of Computer Science
Texas Tech University
Lubbock, Texas, United States
shisawan@ttu.edu

3rd Tanush Arunprasad

Dept. of Computer Science
Texas Tech University
Lubbock, Texas, United States
tarunpra@ttu.edu

4th Jesus Alarcon

Dept. of Computer Science
Texas Tech University
Lubbock, Texas, United States
jesalarc@ttu.edu

5th Gabriel Sotelo

Dept. of Computer Science
Texas Tech University
Lubbock, Texas, United States
gabrsote@ttu.edu

Abstract— The rapid advancement of AI-generated media has made deepfake videos indistinguishable from authentic content, posing serious threats to digital trust, security, and legal integrity. Existing deepfake detection systems are often limited by frame-level analysis and lack of interpretability. This paper presents an explainable deepfake detection framework that leverages temporal modeling to overcome these limitations. The proposed system integrates FFmpeg-based video normalization, RetinaFace-based facial detection, ResNeXt-50 feature extraction, and CNN-based temporal aggregation to classify videos as real or fake. Additionally, an explainability module identifies specific frames contributing to the final prediction, generating forensic-style reports in PDF format. By combining temporal reasoning with interpretable outputs, this work aims to improve the reliability and transparency of deepfake detection systems.

I. INTRODUCTION

The emergence of deep learning-based generative models has led to a new class of digital threats known as deepfakes, where a person's face or identity is manipulated in video content using techniques such as Generative Adversarial Networks (GANs). These synthetic media have been used in political misinformation campaigns, financial fraud, and manipulation of digital evidence, raising serious concerns about trust in online content.

Early deepfake detection methods relied on identifying visual artifacts such as unnatural blinking, boundary inconsistencies, or lighting mismatches. However, recent advances in generative models have significantly reduced these artifacts, making detection increasingly difficult, even for automated systems.

Despite growing research in this area, many existing detection approaches remain limited. Many systems operate at the frame level, treating each frame as an independent image and ignoring temporal relationships. Additionally, most models function as black boxes, providing only a single prediction score without explaining the reasoning behind the decision. These limitations reduce both the robustness and trustworthiness of current detection systems. To address these challenges, this project proposes a deepfake detection framework that emphasizes temporal reasoning and explainability. By analyzing inter-frame differences rather than isolated images, the system captures temporal inconsistencies that are difficult for

generative models to reproduce consistently. Furthermore, the modular design allows individual components to be updated independently, and the explainability module provides interpretable outputs suitable for high-stakes applications.

II. EXISTING PROBLEMS

A. Limitations of Frame-Level Detection

Most existing deepfake detection methods rely on frame-level classification, where individual video frames are analyzed independently using spatial features. While computationally efficient, such approaches inherently ignore temporal dependencies between consecutive frames. As modern deepfake generation techniques continue to improve, they can produce highly realistic individual frames, making purely spatial detection increasingly unreliable.

Recent studies show that frame-level detectors tend to learn dataset-specific artifacts, such as blending boundaries or compression noise, rather than intrinsic forgery characteristics, resulting in poor generalization across datasets [1]. Furthermore, these methods fail to capture temporal inconsistencies such as unstable facial boundaries, inconsistent illumination, and irregular motion patterns that frequently occur in manipulated videos but are difficult for generative models to reproduce consistently over time [1].

Importantly, the effectiveness of incorporating temporal information has been empirically validated. Rowan and Pears demonstrated that models leveraging temporal dependencies achieve statistically significant improvements over frame-dependent approaches ($p < 0.05$), highlighting the importance of spatial-temporal feature learning in deepfake detection [2]. The lack of temporal modeling therefore limits the ability of frame-based approaches to exploit motion coherence and dynamic facial behavior, which are critical cues for distinguishing real from manipulated content.

B. Lack of Interpretability in Predictions

Many deepfake detection systems operate as black-box models, providing only a single prediction score indicating whether a video is real or fake. While effective for classification, such outputs lack transparency and do not provide insight into the reasoning behind the model's decision.

Ensuring explainability remains a primary challenge in deepfake detection systems, as most models fail to provide interpretable justifications for their predictions. This limitation restricts the ability of users to understand which features or regions of the input contributed to the final decision.

Without clear reasoning and visual evidence, users cannot effectively validate model outputs, leading to reduced trust and adoption. Consequently, improving explainability is essential for developing reliable and trustworthy deepfake detection frameworks.

III. SOCIAL IMPACT AND ETHICAL MOTIVATION

Deepfake technology’s misuse is already causing concrete harm. For example, a Hong Kong finance worker was tricked by a deepfake video call of his company’s CFO into transferring over \$25 million to fraudsters [3]. Industry reports further highlight the growing scale of this threat, noting a rise in AI-driven identity fraud and deepfake-enabled scams across financial and corporate sectors [3]. Such realistic forgeries can mimic trusted individuals, making even cautious employees vulnerable. These incidents illustrate that deepfakes pose immediate economic risks by enabling large-scale scams and extortion.

Meanwhile, deepfakes are amplifying misinformation and manipulating public opinion. In a 2025 consumer survey, 68% of respondents said they worried about deepfakes’ societal impact, with “fake news” and misinformation cited as their top concern [4]. Political campaigns have already deployed AI-generated videos, and analysts warn that such ads can leave voters confused or even deceived [5]. Experiments confirm that people often fail to recognize these forgeries, allowing deepfake ads to sway opinions [6].

Deepfakes also threaten the justice system. Courts have already seen AI-generated videos submitted as evidence, prompting concern among judges that they may not reliably distinguish fakery [7]. Legal leaders note that if litigants fear any evidence might be AI-altered, public trust in judicial outcomes is jeopardized.

These threats motivate our project to build an accessible, open-source deepfake detection system. By making the tool public, we enable researchers, journalists, and legal professionals to verify media authenticity. Crucially, our detector emphasizes explainability. Explainable AI is especially important in high-stakes contests, where automated flags must be transparent and contestable by human experts. We also prioritize accuracy and robustness to minimize false positives and false negatives. In this way, our design provides interpretability and emphasizes human oversight, so that users can understand, trust, and fairly evaluate each result rather than treating the system as an infallible judge.

IV. PROPOSED SOLUTION

The proposed deepfake detection system is designed as a modular, multi-stage pipeline that analyzes videos and produces interpretable outputs. Unlike conventional approaches that classify individual frames independently, the system focuses on temporal consistency across sequences of frames, enabling it to

detect subtle artifacts that emerge over time rather than within a single image.

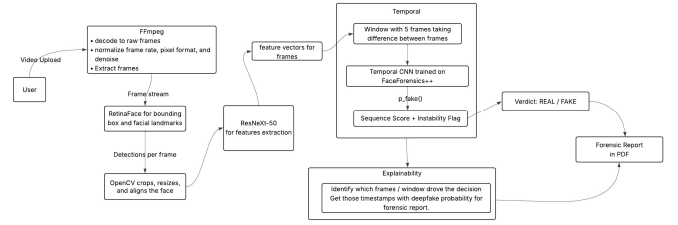


Figure 1: System Architecture

The first stage performs video preprocessing using the FFmpeg framework. Uploaded videos are decoded and normalized to a consistent frame rate, resolution, and pixel format. Additionally, light denoising is applied to reduce compression artifacts introduced by different platforms. This normalization step is crucial to ensure that downstream models receive standardized inputs, thereby improving robustness and reducing variance caused by heterogeneous data sources.

Following preprocessing, individual frames are passed to a face detection module based on RetinaFace, which generates bounding boxes and facial landmarks for all detected faces. These landmarks are then used in conjunction with OpenCV to crop, resize, and geometrically align each face. Alignment ensures that facial regions are consistently positioned across frames, minimizing geometric noise such as rotation or scale variations. This step is essential because the downstream feature extractor is sensitive to spatial inconsistencies that are unrelated to deepfake artifacts.

The aligned face crops are then passed into a ResNeXt-50 model [8] without the classification head, which generates high dimensional feature embeddings for each frame. These embeddings capture rich visual information, including facial texture, structural geometry, and bounding coherence. By leveraging a pretrained convolutional architecture, the system extracts semantically meaningful representations that are more robust than raw pixel inputs.

To model temporal relationships, the extracted features are fed into a Temporal CNN-based module. Rather than passing raw embeddings directly, the system computes the difference between consecutive frame embeddings within a window of five frames, producing four difference vectors per window. These difference vectors explicitly represent the change in visual features between adjacent frames, making the model more sensitive to temporal inconsistencies that are characteristic of manipulated content. The difference vectors are transposed and treated as a one-dimensional signal, with the 2048 feature dimensions acting as channels and the four time steps as the sequence. They are then passed through a Conv1D branch with a kernel of size 3, ReLU activation, Batch Normalization, and Global Average Pooling. The resulting representation is passed through a two-layer fully connected classifier with dropout to produce a window-level manipulation probability. This design is computationally lightweight while remaining temporal in reasoning, since it processes only changes between frames rather than the absolute frames.

The system then aggregates probability scores across all windows to produce a final video-level classification using a simple mean. In addition to generating a binary real/fake decision, the model tracks which frame windows contribute most significantly to the prediction. These contributions are compiled in an explainability module, which generates a structured report highlighting suspicious timestamps.

The final output of the system includes a classification label (real or fake), a confidence score, and a set of flagged timestamps indicating regions of interest. The output can be exported in PDF format, ensuring accessibility for both technical users and non-technical stakeholders.

V. TESTING, EVALUATION, AND BENCHMARKING

To rigorously evaluate the effectiveness of the proposed deepfake detection system, we assess its performance using both quantitative and comparative benchmarks.

A. Dataset and Experiment Setup

The system is trained and evaluated using the FaceForensics++ (C23 compression) dataset, accessed via Kaggle. The dataset contains 1,000 original videos and 1,000 Deepfake videos generated using the autoencoder based face-swap technique. All videos were preprocessed through the same pipeline described in the Section IV: FFmpeg decoding at 10 FPS with light denoising, RetinaFace face detection, OpenCV-based cropping and geometric alignment, and ResNeXt-50 feature extraction (final classification head was removed, producing 2048-dimensional embeddings per frame). These per-frame embeddings were saved as NumPy arrays and treated as our new dataset to train and test the models.



Figure 2: Dataset preparation for training

The resulting feature vectors were split at the video level into 80% training, 10% validation, and 10% test sets. Critically, the same split was used for training and testing both our temporal based CNN model as well as baseline model.

B. Evaluation metrics

While overall accuracy provides a basic measure of performance, it is insufficient for high-stakes applications such as fraud detection and legal verification. Therefore, we evaluated the models using multiple metrics as listed below.

- **Accuracy:** Measures the overall proportion of correctly classified videos.
- **AUC-ROC (Area Under the Receiver Operating Characteristic Curve):** Measures models' ability to separate real and fake videos.
- **Precision:** Evaluates how many videos classified as deepfakes are actually fake, which is critical for minimizing false accusations.

- **Recall:** Measures how many actual fake videos are correctly identified, which is critical for minimizing missed detections.
- **F1-score:** F1-score is the harmonic mean of precision and recall, providing a balanced single metric.
- **AUC-PR (Area Under the Precision-Recall Curve):** A high AUC-PR indicates that model can identify most deepfakes (Recall) while maintaining a low rate of false alarms (Precision).
- **False Positive Rate (FPR):** Indicates how often real videos are incorrectly flagged as fake.
- **False Negative Rate (FNR):** Indicates how often deepfakes are missed by the system.

To handle false predictions, the system outputs confidence scores for each classification. Predictions with low confidence or those near the decision boundary are flagged for manual review. This reduces the risk of critical misclassifications, particularly in high-stakes scenarios such as legal verification.

C. Baseline Comparison

The proposed temporal CNN was compared against a frame-level MLP baseline designed for direct and fair comparison. For fairness in comparison, instead of external third-party model, this baseline was trained from scratch on the same dataset, using the same feature extraction pipeline and the same train/validation/test split. This approach was chosen to eliminate factors such as difference in preprocessing, data distribution, or training data. The MLP baseline (SimpleFrameMLP) takes individual 2048-dimensional frame embeddings as input and passes them through two fully-connected layers (2048->256->1) with ReLU activation, followed by a sigmoid output. In this way, each frame was classified independently, and the video-level prediction was obtained by mean pooling the frame-level probabilities. For both the temporal based CNN model and baseline model, videos with deepfake probabilities of values greater than or equal to 0.50 were considered as deepfake and less than 0.50 were considered real to compare the metrics.

The following table presents the test-set performance of both models at the video level.

TABLE 1: Baseline Comparison Results

Metric	Temporal CNN	Baseline
Accuracy	90.00%	88.00%
Precision	0.8774	0.8958
Recall	0.9300	0.8600
F1-score	0.9029	0.8776
AUC-ROC	0.9374	0.9315
AUC-PR	0.9381	0.9435
FPR	0.1300	0.1000
FNR	0.0700	0.1400

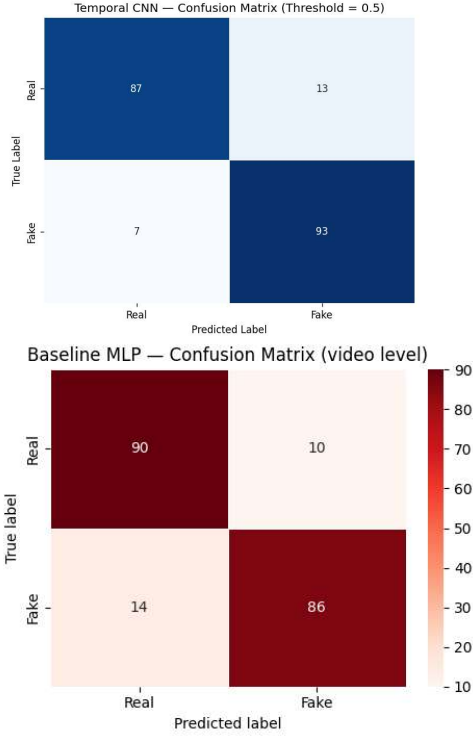


Figure 3: Confusion matrix- CNN(top) & Baseline(bottom)

The temporal based CNN outperforms the baseline frame-based model on accuracy (90.00% vs. 88.00%), recall (0.93 vs. 0.86), F1-score (0.9029 vs. 0.8776), and AUC-ROC (0.9374 vs. 0.9315), demonstrating that modeling inter-frame differences provided a meaningful improvement in deepfake detection capability. The higher recall of the temporal model indicates that it is significantly better at catching fake videos. Its False Negative Rate is 0.07 compared to 0.14 for the baseline model which is important in high-stakes applications such as legal verification and investigative journalism.

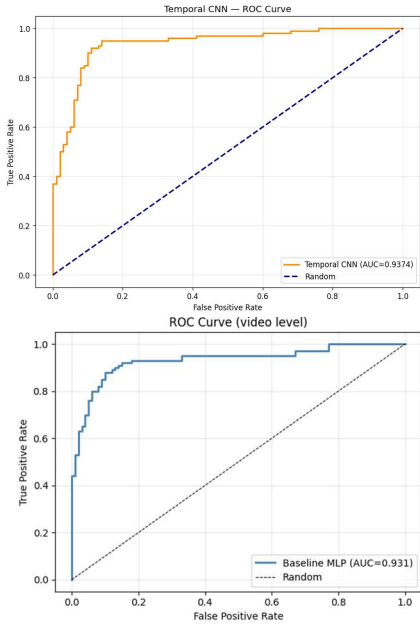


Figure 4: AUC-ROC for CNN(top) & Baseline(bottom)

The baseline model outperforms the temporal based CNN model in terms of False Positive Rate (0.10 vs. 0.13), meaning it incorrectly flags fewer real videos as fake. Also, the baseline model has a higher AUC-PR (0.9435 vs. 0.9381) which suggests that it produces comparatively fewer false positives when identifying the deepfakes compared to the CNN model. Also, the baseline model achieves higher precision (0.8958 vs. 0.8774), which suggests that baseline requires stronger frame-based signal before committing to a fake prediction. As a result, it produces fewer false positives (0.10 vs. 0.13). On contrast, the temporal based model seemed to be more sensitive and more likely to flag real videos as suspicious. This precision gap tells us that temporal based models that are sensitive to inter-frame variation may also be triggered by natural motion artifacts or compression noise in real videos, suggesting that post-processing or experimenting different decision threshold (currently 0.5) could help narrow this gap in future works.

D. Limitations, Generalization, and Future Work

The proposed system is trained and evaluated on a subset of the FaceForensics++ dataset consisting of 1000 original and 1000 Deepfake videos, where the Deepfake category represents a single manipulation method: autoencoder-based face-swapping. While computationally efficient for training and testing, it introduces two limitations.

First, the dataset scope is narrow. FaceForensics++ contains multiple manipulation methods such as Face2Face, FaceSwap, and NaturalTextures. Our system was trained exclusively on one face swap method due to limitations in compute resources and may have learned features specific to that generation technique rather than general indicators of temporal inconsistency, limiting its ability to generalize to unseen manipulation types. Evaluation on a more diverse dataset and cross-dataset evaluation is necessary to test true generalization which we have planned for future works.

Second, we found that the quality of the deepfakes in this particular subset was mixed. Some inspection revealed that many of the manipulated videos contain visible artifacts such as unnatural boundary blending that are detectable easily even by human eyes. This raises an important question about the interpretation of both models' results. If significant portion of the fake videos were easily distinguishable from original videos on the basis of individual frames alone, a frame-level model may achieve competitive performance not by generalizing features but by exploiting frame based artifacts specific to the dataset. This could also partly explain why the baseline performs comparable to the temporal based CNN on most metrics. Conversely, this observation may not hold true under rigorous analysis i.e. the frame-based model could have actual discriminative power. This ambiguity further motivates our future works including evaluation on higher-quality and more recent deepfakes generated with diverse manipulation techniques.

Deepfake generation techniques continue to evolve, particularly with the rise of adversarial methods designed to bypass standard classifiers, making robust detection a significant challenge. By employing temporal modeling and feature-level analysis, the system aims to increase the difficulty for generators to maintain consistency over frames and reduce

reliance on pixel-level artifacts that can be easily manipulated. The results from our experiments showed that temporal modeling offers meaningful improvements over frame-based technique and the gap may widen on higher-quality deepfake datasets.

Future work for this system includes evaluation on additional FaceForensics++ manipulation types and external datasets to improve and test cross-dataset generalization. Also, future works could include experimenting with longer temporal windows, overlapping windows, and even threshold calibration to improve precision-recall tradeoff without retraining.

VI. ALTERNATIVE APPROACHES AND EXTENSIONS

The proposed system uses a Temporal based CNN operating on inter-frame difference vectors, but it is not the only available approach for this task. One potential alternative is LSTM networks. Similarly, alternative approaches could include experimenting with different window sizes as well. Similarly, Gated Recurrent Unit (GRU) is another alternative, which simplifies the LSTM architecture by reducing the number of gating mechanisms while achieving comparable performance. For example, R. Hande et al. directly compared CNN+LSTM and CNN+GRU pipeline to analyze sequential data and found that both architectures achieved competitive results [11]. Further, we could look into diverse models and create an ensemble of different models to evaluate whether their combined outputs yield better accuracy in detecting instability over single models.

VII. SYSTEM ANALYSIS AND DECOMPOSITION

The deepfake detection system is decomposed into nine distinct modules, each responsible for a well-defined stage of the detection pipeline. This modular architecture promotes separation of concerns, enabling individual components to be developed, tested, and updated independently.

A. WebInterface

The WebInterface is the front-facing module that allows users to interact with the system even without any programming knowledge. It provides a video upload form, displays the final verdict and confidence score, and allows users to jump to a specific timestamp in the uploaded video and analyze the probability of that part of the video being deepfake as output by the system. Users can also download the generated report in PDF format from this module.

B. VideoInputHandler

The VideoInputHandler serves as the entry point to the system, responsible for receiving user-submitted video files through the web interface. It validates the uploaded file format and size, and then passes the raw video to the preprocessing stage. This module abstracts the file I/O for the rest of the pipeline, ensuring that the downstream modules always receive a video in a valid format.

C. FFmpegPreprocessor

The FFmpegPreprocessor normalizes the incoming video to a consistent format before any analysis takes place. It decodes the video into individual frames, resamples to a target frame rate, resizes to a standard resolution, and applies light denoising to

reduce compression artifacts introduced by different platforms. This normalization is critical because the downstream models are sensitive to resolution and quality inconsistencies that are unrelated to deepfake artifacts. FFmpeg is used as the underlying processing engine due to its broad codec support and efficiency.

D. FaceDetection

The FaceDetection module applies the RetinaFace model to each preprocessed frame to locate all human faces present. It produces bounding boxes defining the region of each detected face and extracts five-point facial landmarks (eyes, nose, and mouth corners) per detection. These landmarks are passed to the next module.

E. FaceCropAndAlignment

The FaceCropAndAlignment module takes the bounding boxes and landmarks from the FaceDetection module and produces a geometrically standardized face crop for each detection. It uses OpenCV to crop the face region from the full frame and apply transformation using the landmarks to normalize rotation, scale, and translation. Consistent alignment across frames is essential because the ResNeXt-50 model in feature extractor is sensitive to geometric variations unrelated to manipulation artifacts.

F. FeatureExtractor

The FeatureExtractor module passes each aligned face crop through a pretrained ResNeXt-50 convolutional neural network to produce a high-dimensional feature embedding. The ResNeXt-50 architecture, pretrained on ImageNet, captures rich spatial representations including facial texture, structural geometry, and boundary coherence. The final layer of the model is adapted to output a fixed-length embedding vector that encodes the visual characteristics of each individual face crop. These vectors are passed to the temporal modeling stage.

G. TemporalAnalyzer

The TemporalAnalyzer is the core detection module. Rather than classifying individual frames, it analyzes sequences of feature vectors using a Temporal CNN trained on the FaceForensics++ dataset. A non-overlapping sliding window groups consecutive frame embeddings into window of 5 frames. For each window, the difference vectors between the consecutive embeddings are computed, producing 4 difference vectors of dimension 2048. These are transposed to shape (2048, 4) and passed through the model to produce manipulation probability for each window, which is later combined to produce a video-level verdict using a simple mean.

H. ExplainabilityModule

The ExplainabilityModule transforms the raw probability scores produced by the TemporalAnalyzer into interpretable, human-readable evidence. It maps each window's decision back to the corresponding frame indices in the original video, flags temporal segments that exceed a suspicion threshold, and produces a probability distribution over time. This allows users to identify exactly which portions of the video drove the model's decision, rather than only a binary verdict.

I. ReportGenerator

The ReportGenerator compiles the verdict, confidence score, flagged timestamps, and probability distribution graph produced by the ExplainabilityModule into a structured, shareable report. Reports are generated in PDF format.

VIII. UML DIAGRAMS

A. Use Case Diagram

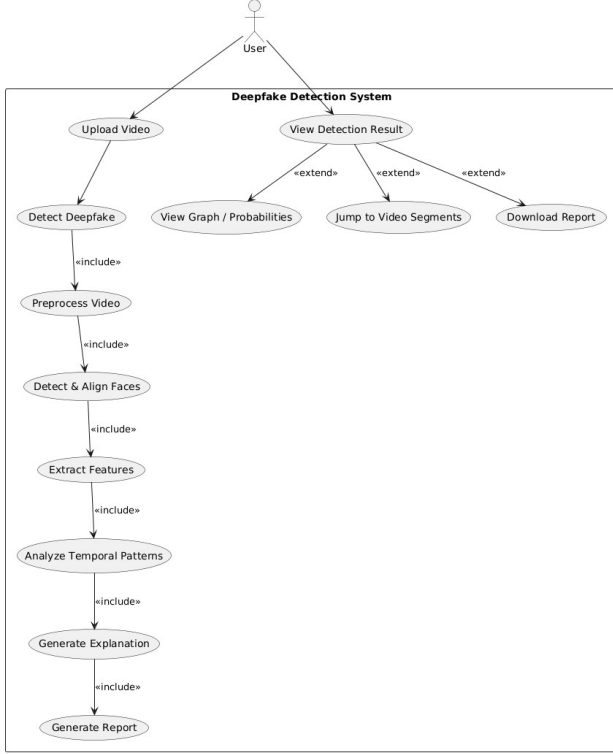


Figure 5: Use Case Diagram

The use case diagram captures the behavioral view of the system from the perspective of its primary actor, User, who is any individual interacting with the system through the web interface. The user directly initiates two interactions: Upload Video, which triggers the entire detection pipeline, and View Detection Result, which presents the outcome once the processing is complete. All the internal processing is encapsulated within the Detect Deepfake use case, which the system executes automatically upon a successful upload.

The internal pipeline is modeled as a chain of `<<include>>` relationships under Detect Deepfake. Each of the preprocessing, face detection and alignment, feature extraction, temporal analysis, explanation generation, and report generation is mandatory. The `<<include>>` relationship reflects that none of these stages are optional, and skipping any one would produce an invalid or incomplete result.

The View Detection Result has three `<<extend>>` relationships with View Graph / Probabilities, Jump to Video Segments, and Download Report, representing optional actions the user may take after a result is available.

B. Class Diagram

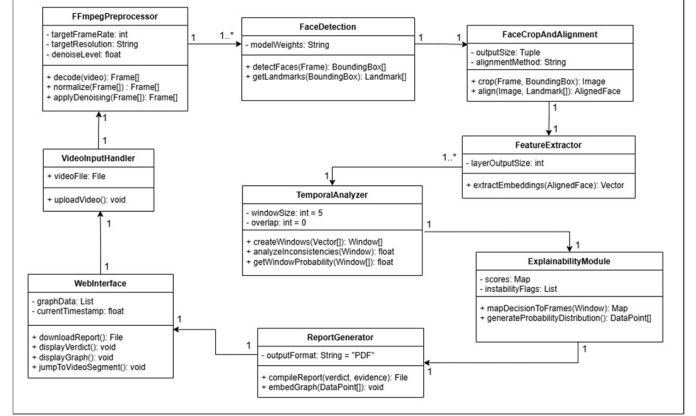


Figure 6: Class Diagram

The class diagram above presents the structural design of the deepfake detection system, illustrating the nine core classes, their attributes and methods, and the associations between them. Each class corresponds directly to a module in the pipeline as described in the System Analysis and Decomposition section.

1) *Class Descriptions*: The WebInterface class holds graphData attribute which is a time-series data for timeline visualization, and currentTimestamp attribute which tracks the currently viewed position in the video. The downloadReport() method triggers PDF report download, displayVerdict() method renders the classification result, displayGraph() method renders the probability distribution chart, and jumpToVideoSegment() method seeks the video player to a flagged timestamp.

The VideoInputHandler class holds videoFile attribute which is the raw video object uploaded by the user, and uploadVideo() method which triggers the pipeline after validating the file. The FFmpegPreprocessor class holds the targetFrame, targetResolution, and denoiseLevel attributes to define the normalization parameters. It also has decode(video) method to decode the video into a frame array, normalize(Frame[]) method to standardize frame rate and resolution, and applyDenoising(Frame[]) method to reduce encoding noise.

The FaceDetection class holds modelWeights attribute which represents the path to the pretrained RetinaFace model weights, detectFaces(Frame) method that returns bounding boxes for all detected faces, and getLandmarks(BoundingBox) method that extracts facial key points within a detected region. The FaceCropAndAlignment class holds outputSize attribute representing the target pixel dimensions of the aligned face crop and alignmentMethod attribute representing geometric transformation strategy. Also, it has crop(Frame, BoundingBox) method that isolates the face region and align(Image, Landmark[]) method that applies geometric correction to standardize pose.

The FeatureExtractor class has layerOutputSize attribute representing the dimensionality of the output embedding vector and extractEmbeddings(AlignedFace) method that runs the forward pass through the ResNeXt-50 and returns the feature vector. The TemporalAnalyzer class has windowSize attribute

that represents number of frames per temporal window and overlap attribute that represents frame overlap between consecutive windows. Also, it has `createSlidingWindow(Vector[])` method that partitions the sequence of embeddings into overlapping windows, `analyzeInconsistencies(Window)` method that runs the Temporal CNN on a single window of difference vectors and returns a deepfake probability, and `getSequenceProbability(Window[])` method that aggregates window-level scores into a final video-level score.

The `ExplainabilityModule` class has `scores` attribute that maps window indices to their deepfake probability scores, `instabilityFlags` attribute that represents the list of flagged frame ranges that exceed the suspicion threshold. Also, it has `mapDecisionToFrames(Window)` method that associates each window's score with its corresponding frame indices and `generateProbabilityDistribution()` method that produces a time-series dataset of per-window scores for visualization. The `ReportGenerator` class has `outputFormat` attribute that specifies the default export format which is PDF by default. Also, the class has `compileReport(verdict, evidence)` method that assembles all results into a structured output file and `embedGraph(DataPoint[])` method that inserts the probability graph into the report.

2) *Class Relationships*: The `VideoInputHandler` and `FFmpegPreprocessor` share a 1-to-1 association because each video upload triggers exactly one preprocessing task, and there is no scenario where a single upload triggers multiple preprocessors. This keeps the pipeline straightforward and simple per video upload.

The `FFmpegPreprocessor` and `FaceDetection` share a 1-to-1..*, meaning one `FFmpeg` preprocessing task produces one or more frames that are passed to `FaceDetection` class. The multiplicity on the `FaceDetection` side is 1..* rather than exactly 1 because a video always produces at least one frame, and usually many. This reflects the reality that the preprocessor outputs a frame array, not a single object.

`FaceDetection` and `FaceCropAndAlignment` share a 1-to-1 association. While `FaceDetection` may return multiple bounding boxes per frame, the crop and alignment operation is applied uniformly to each detection in sequence. The two classes are closely related in the design, and every detection must be aligned before feature extraction, so they have a 1-to-1 relationship.

`FaceCropAndAlignment` and `FeatureExtractor` also share a 1-to-1 association because each aligned face crop is passed directly to the feature extractor, which processes it and returns exactly one embedding vector.

`FeatureExtractor` and `TemporalAnalyzer` share a 1..* to 1 association. This is because temporal analyzer receives the full sequence of embedding vectors produced across all frames before it begins processing, and a single `TemporalAnalyzer` instance must receive embeddings from multiple frames to form even one sliding window.

`TemporalAnalyzer` and `ExplainabilityModule` share a 1-to-1 association because the `TemporalAnalyzer` produces one set of window-level probability scores per video, which is then passed to the explainability module.

`ExplainabilityModule` and `ReportGenerator` share a 1-to-1 association because the structured evidence produced by the explainability module including the score map, flagged frame ranges, and probability distribution is passed to `ReportGenerator` in a single operation.

`ReportGenerator` and `WebInterface` share a 1-to-1 association since exactly one compiled report is delivered to the web interface for display and download at a time. Finally, `WebInterface` closes the loop back to `VideoInputHandler` with a 1-to-1 association, representing the user's ability to submit a new video from the same interface.

C. Sequence Diagram

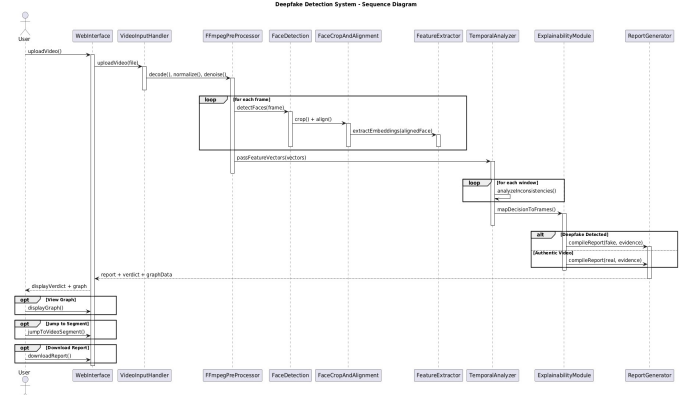


Figure 7: Sequence Diagram

The sequence diagram illustrates the ordering of interactions between system components for a single video submission, from upload through result delivery. The `WebInterface` participates only at the boundaries of the interaction when initiating the pipeline on upload and receiving the final output.

The per-frame loop shows that `FaceDetection`, `FaceCropAndAlignment`, and `FeatureExtractor` modules operate synchronously on one frame at a time and generate feature vectors that are then passed to the `TemporalAnalyzer`. In the `TemporalAnalyzer`, the internal loop operates on sliding window of frames rather than individual frames which reflects the difference in our design from the popular frame-level prediction designs.

The activation bars in the diagram represent the duration for which a component is actively performing an operation or awaiting a response. For example, the `WebInterface` remains active throughout the interaction whereas the internal processing components deactivate after completing the designated tasks and thus have shorter activation bars.

The alt block at the `ExplainabilityModule` stage represents two possible outcomes for the final verdict i.e. real or fake. Regardless of the outcome, a report is always generated. Finally, the three opt fragments at the end capture the optional user-driven interactions such as viewing graphs, navigating to specific video segments, or downloading the generated report, which occur after the processing pipeline has completed.

IX. ALGORITHMS

The deepfake detection system relies on two core algorithms that work together within the pipeline as described below in the flowchart diagrams.

A. Temporal Modeling

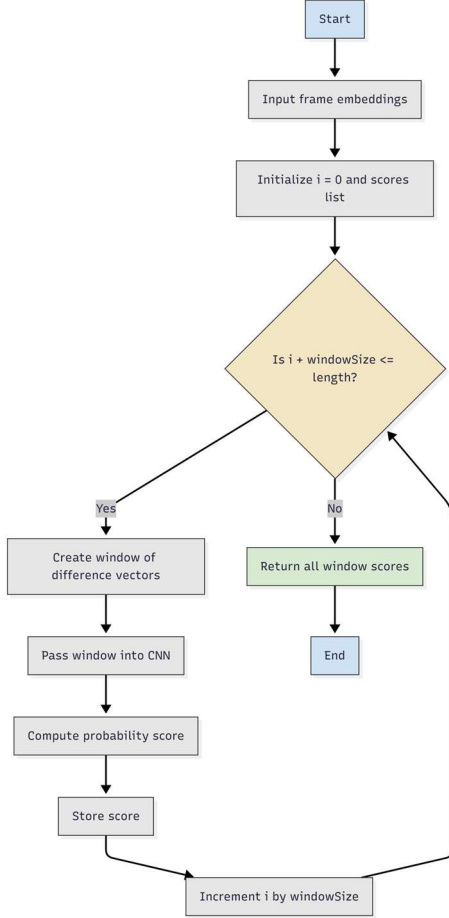


Figure 8: Temporal Modeling Algorithm

The Temporal Modeling Algorithm operates at the TemporalAnalyzer module in our system and is responsible for taking the embedding vectors produced by the FeatureExtractor as input and producing a mapping from window of frames to their respective probability of being deepfake as detected by our model as output. The algorithm starts with the list of embedding vectors and creates a window of 5 frames at a time if there are sufficient frames. Then, it creates a window of 4 difference vectors and passes it into our Temporal CNN trained on the FaceForensics++ dataset, which analyzes the temporal differences between frames and outputs a score that represents the probability that the frames have been manipulated. Then, the algorithm stores the score for the window and moves to the next window. This process is repeated as long as there are sufficient embedding vectors to create windows and the algorithm finally outputs a mapping from windows of frames to their respective probabilities of being deepfake.

B. Explainability / Frame Flagging

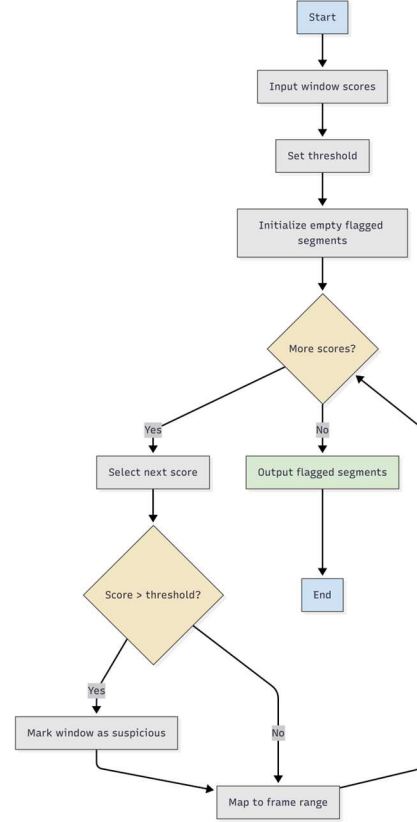


Figure 9: Explainability / Frame Flagging Algorithm

The Frame Flagging Algorithm compares the manipulation scores for the windows against a threshold value to mark a window as suspicious or not. This algorithm also maps the scores back to its corresponding frame indices in the original video.

X. SYSTEM INTERFACE

The system provides a web-based interface that allows users to upload a video, view the detection results, and download a structured report, all even without any programming knowledge. The interface is divided into four main views.

A. Upload Screen

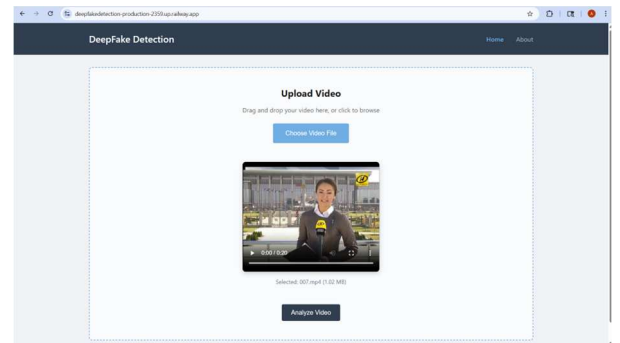


Figure 10: Upload Screen

The upload screen is the entry point of the application. It presents the user with a “Choose Video File” button that opens the computer’s file explorer, allowing selection of a video file.

Once the file is selected, the user submits it by clicking the “Analyze Video” button, which sends the video to the system’s backend for processing video through the detection pipeline.

B. Results Screen

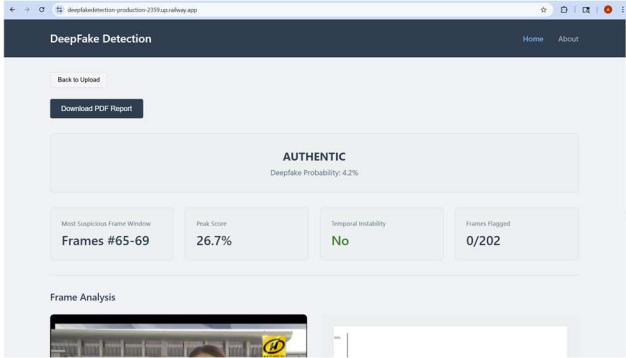


Figure 11: Results UI View

After processing is complete, the results screen displays a summary of the detection outcome. The verdict label indicates whether the video is classified as real or likely manipulated, accompanied by an overall probability score. Below the verdict, the screen reports the most suspicious frame window i.e. the temporal segment with the highest manipulation probability, along with the Peak Score. Additional fields include Temporal Instability, which indicates whether the model detected temporal inconsistencies across windows, and Frames Flagged, which reports how many frame windows exceeded the suspicion threshold out of the total processed.

C. Video Player and Bar Graph

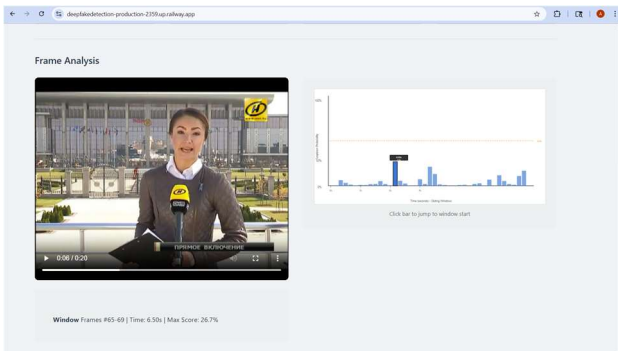


Figure 12: Video Player and Interactive Chart

This view displays the uploaded video alongside an interactive bar chart. Each bar in the chart represents one frame window, and the y-axis shows the manipulation probability score for that window. Clicking on any bar in the chart automatically seeks the video player to the corresponding timestamp, allowing the user to directly inspect the portion of the video the model flagged as suspicious. This interaction ties the output of the model directly to the visual content, aiming to make it easier to evaluate the model’s reasoning.

D. PDF Report

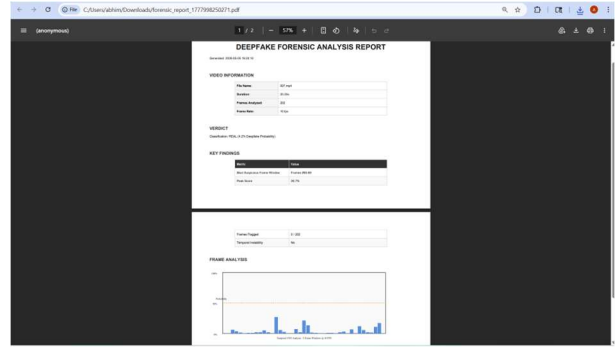


Figure 13: PDF Report

When the “Download PDF Report” button is clicked, the system generates a formatted PDF document containing all the information shown on the results screen. The report is designed to be shareable with stakeholders such as legal professionals or journalists who may need a documented record of the analysis.

XI. IMPLEMENTATION DETAILS

A. Implementation Overview

The deepfake detection system was implemented as a full-stack application comprising a React-based frontend, a FastAPI backend, and a PyTorch inference pipeline. The frontend provides a web interface for video upload, result visualization and report download as described in Section X, while the backend exposes REST API endpoints that handle the detection pipeline.

All model training was conducted on Google Colab using GPU-accelerated environments, with feature vectors and model checkpoints stored persistently on Google Drive. During development and demonstration, the backend was hosted on Google Colab itself and endpoint was exposed via Ngrok tunneling, with frontend deployed on a web hosting platform called railway. While not a production-grade deployment strategy, this arrangement provided us sufficient infrastructure for end-to-end testing and demonstration.

B. Key Implementation Techniques

For dataset preparation, 2000 videos from FaceForensics++ dataset was preprocessed across five batched Kaggle sessions to stay within Kaggle’s GPU session time limits. Each video was processed through the preprocessing pipeline till the feature extraction module to get our new dataset as described in Section V. The final output was 2048-dimensional embeddings per frame stored as NumPy arrays. This converted the problem from raw video processing into sequence modeling on fixed dimensional feature vectors, allowing us to train and test models independently of the computationally expensive video preprocessing stage. We used the same dataset with exact same split for training and testing both temporal based model as well as the baseline model for fairness. For the temporal CNN, the feature vectors were transposed to (2048, 4) shape taking only the difference vectors and processed through a Conv1D branch with kernel size 3, Batch Normalization, ReLU activation, and

Global Average Pooling, followed by a two-layer fully connected classifier with dropout.

C. Completed and Remaining Functionalities

The core system functionality has been fully implemented, including the end-to-end video processing pipeline, the explainability module, the interactive web interface with synchronized video player and probability bar chart, and forensic PDF report generation.

The system was validated through informal testing with users outside the development team, including people unfamiliar with the underlying machine learning pipeline. Testers were asked to upload videos through the web interface and evaluate whether the system’s outputs were interpretable without any technical guidance. Feedbacks confirmed that the interface was accessible to non-technical users and that the forensic PDF report provided sufficient context for understanding the detection reasoning.

Remaining work is primarily focused on generalization and robustness of the system. This includes cross-dataset evaluation on additional FaceForensics++ manipulation types and other external datasets, exploration of other temporal architectures such as LSTM and GRU, experimentation with decision threshold (currently 0.50) to improve precision-recall tradeoff, and real-time streaming inference for live video analysis.

D. Implementation and Technical Challenges

We encountered several non-trivial challenges during development. The most significant constraint was computational infrastructure. Processing 2000 videos through the full preprocessing pipeline in Kaggle approached the allowed GPU session limits, requiring us to split the work across five sessions. During deployment, running RetinaFace and ResNeXt-50 simultaneously exceeded the RAM capacity on our personal laptops that had CPUs only. Even with GPU acceleration on personal laptops, the end-to-end processing of 20 seconds video took approximately two to three minutes which seemed too slow for a seamless interactive experience. These constraints motivated us to use Google Colab as the hosting environment for testing and demonstration by using tunneling with Ngrok.

Model development also presented its own challenges. Early iterations of the temporal CNN with larger parameters achieved over 99% training accuracy but dropped to 65-70% on validation and testing accuracy, indicating that the model was memorizing rather than generalizing. This was a direct consequence of the relatively small dataset size of only 2000 videos, which is quite below the scale typically used in deepfake detection research. We addressed such challenges by reducing the model capacity. Also, window size was fixed at five frames with no overlaps after experiments showed that larger windows and overlaps increased redundancy without improving performance.

The original plan for the baseline model was to use a publicly available pretrained deepfake detector from Hugging Face [10]. However, this model and its variants produced near-random predictions on the test dataset. This motivated us to train the MLP based baseline model from scratch on same data and splits. We believe that this choice ultimately produced a more

meaningful and rigorous comparison, since all variables other than the temporal modeling were kept constant for fairness.

XII. TECHNOLOGIES

The system is built using a combination of frontend, backend, and machine learning libraries, each chosen to handle a specific role in the pipeline. The frontend is implemented in React, using Axios for HTTP communication with the backend and Chart.js for the interactive bar chart visualization. The backend is built on FastAPI, a Python web framework. It handles video uploads, manages the detection pipeline, and returns forensic results to the frontend. Core pipeline dependencies include FFmpeg for video decoding, frame normalization, and denoising; RetinaFace for face detection and landmark extraction; OpenCV for geometric face alignment; PyTorch for running the ResNeXt-50 feature extractor and the temporal CNN; and NumPy for array operations throughout the pipeline. PDF report generation is handled by a Python library named ReportLab.

XIII. LOCAL AND GLOBAL IMPACTS

A. Impact on Individuals

Deepfake technology poses direct and personal threats to individuals. Non-consensual synthetic media where a person’s face is placed into manipulated videos without their knowledge have been used for harassment, reputational damage, and intimidated image abuse. Victims often have limited legal recourse and face significant psychological harm. Detection systems like the one developed in this project provide a mechanism for individuals to verify whether video content including them or others has been manipulated, offering a practical means of self-protection and evidence gathering.

For everyday users, the proposed system’s emphasis on explainability is particularly significant. Rather than returning a black-box verdict, the system identifies which parts of the video drove the prediction and presents this evidence in a structured, downloadable PDF report. The system is also designed to be accessible without programming knowledge through a web-based interface. This design helps both technical and non-technical users to access the reasoning behind the decision rather than being asked to simply trust an automated output. In contexts such as personal legal disputes, employment screening, or online harassment cases, this transparency can make a significant difference.

B. Impact on Organizations

At the organizational level, deepfakes represent a growing area for financial fraud and reputational attack. The widely reported case of a Hong Kong finance worker who was deceived by a deepfake video call into transferring over \$25 million illustrates the scale of risk that even large organizations now face [3]. Detection systems similar to the one we propose in this project, which are modular, deployable, and capable of processing uploaded videos programmatically, can be integrated into enterprise content verification workflows, media authentication pipelines, and compliance systems.

For news organizations and media companies, the explainability module provides a particularly valuable capability. Journalists investigating potentially manipulated

video evidence often require not only a verdict but a documented report. The timestamped forensic PDF report generated by the proposed system is designed especially for this use case. Legal and compliance teams similarly benefit from structured, shareable evidence rather than simple Yes/No verdict.

The open-source orientation of the project further amplifies organizational impact. By making the system publicly available, the project enables smaller organizations, including newsrooms, nonprofits, and civil society groups that lack resources to develop proprietary detection systems, to access the verification capability.

C. Impact on Society

In current society, the widespread use of deepfakes already threatens democratic discourse, the justice system, and public trust in media.

In democratic contexts, manipulated videos have already been deployed in political campaigns, with analysts warning that such content can leave voters confused or deceived [5]. Detection systems of the kind that we propose can be used by election monitoring bodies, fact-checkers, and platform moderators as a countermeasure against coordinated disinformation campaigns, helping preserve the informational integrity on which democratic participation depends. In the justice system, the use of deepfake videos poses a direct threat to the legal proceedings. A detection system with explainable outputs can be used in such contexts, where human experts must be able to interrogate and understand the detected results. Overall, the capacity of a society to distinguish real content from manipulated ones is one of the serious risks today and in future. Projects including the one we propose here, that advance detection capability and emphasize on interpretability, can contribute to rebuilding and maintaining public trust in media.

D. Ethical Considerations

While the deepfake detection system benefits society, the development and deployment of the system also has some ethical considerations.

First, false positives carry real costs. In legal or journalistic contexts, a false accusation of manipulation could harm innocent individuals. The system addresses this through confidence score and flagging low-confidence predictions for manual review, so human oversight may still be needed. Second, the current system is trained exclusively on the autoencoder-based Deepfake subset of FaceForensics++ dataset, and responsible deployment requires ongoing evaluation across diverse datasets to ensure that the system does not perform disproportionately. Third, as the deepfake generators evolve and become more sophisticated, the detection systems should also evolve and no detection system should be treated as a permanent solution. The modular architecture of the proposed system where the preprocessing components, feature extractor, and temporal layer can be updated independently is designed with this in mind to enable the system to evolve alongside the manipulation techniques. Finally, the decision to make the system open-source carries dual use implications. While public availability maximizes benefit to legitimate users, it also makes the system's detection logic exposed to the adversaries seeking to defeat it.

XIV. CONCLUSION

Deepfake videos represent a growing threat to digital trust, public discourse, and legal integrity. This project aims to address two key limitations of existing detection systems: reliance on frame-level analysis and lack of interpretability. The result is a system that not only classifies videos as real or manipulated, but also identifies which parts of the video drove that decision, making it practical for high-stakes use cases such as legal verification and investigative journalism.

Evaluated against a frame-level MLP baseline model trained on identical data and splits, the proposed temporal based CNN achieved 90.00% accuracy and a recall of 0.93, compared to 88.00% accuracy and recall of 0.86 for the baseline. The reduction in false negative from 14% for the baseline to 7% for the CNN model also signifies the importance of the proposed system since missed deepfakes carry greater real world consequences.

Overall, this work demonstrates that explicitly modeling temporal inconsistencies between frames provides meaningful detection improvements over frame-level analysis, and that interpretable outputs are essential for building systems that users can trust.

REFERENCES

- [1] D. A. Coccomini, R. Caldelli, F. Falchi, and C. Gennaro, "On the generalization of deep learning models in video deepfake detection," *J. Imaging*, vol. 9, no. 5, p. 89, May 2023, doi: 10.3390/jimaging9050089.
- [2] W. Rowan and N. Pears, "The effectiveness of temporal dependency in deepfake video detection," arXiv preprint arXiv:2205.06684, 2022. [Online]. Available: <https://arxiv.org/abs/2205.06684>.
- [3] D. Ledet and E. Knott, "Real insurance coverage for increasing AI deepfake risks," Reuters, 11 Apr. 2024. [Online]. Available: <https://www.reuters.com/legal/legalindustry/real-insurance-coverage-increasing-ai-deepfake-risks-2024-04-11>.
- [4] Security Staff, "68% of people are worried about misinformation due to deepfakes," Security Magazine, 24 Feb. 2025. [Online]. Available: <https://www.securitymagazine.com/articles/101414-68-of-people-are-worried-about-misinformation-due-to-deepfakes>.
- [5] J. Ax and H. Coster, "AI deepfakes blur reality in 2026 US midterm campaigns," Reuters, 28 Mar. 2026. [Online]. Available: <https://www.reuters.com/business/media-telecom/ai-deepfakes-blur-reality-2026-us-midterm-campaigns-2026-03-28>.
- [6] N. Naffi, "Deepfakes and the crisis of knowing," UNESCO, 1 Oct. 2025. [Online]. Available: <https://www.unesco.org/en/articles/deepfakes-and-crisis-knowing>.
- [7] C. Heaton, S. Cleary, and M. Navin, "AI-generated evidence is a threat to public trust in the courts," National Center for State Courts, 24 Feb. 2026. [Online]. Available: <https://www.ncsc.org/resources-courts/ai-generated-evidence-threat-public-trust-courts>.
- [8] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He, "Aggregated residual transformations for deep neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 1492–1500.
- [9] A. Rossler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, and M. Nießner, "FaceForensics++: Learning to detect manipulated facial images," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1–11.
- [10] P. Vignesh, "Deepfake Detector Model V1," Hugging Face, 2024. [Online]. Available: <https://huggingface.co/prithivMLmods/deepfake-detector-model-v1>.
- [11] R. Hande, S. Goon, A. Gondhali, and N. Singhaniya, "A Novel Method of Deepfake Detection," *ITM Web Conf.*, Vol. 44, p. 03064, 2022, doi: 10.1051/itmconf/2022440306.